



IP INFO WIDGET

Project Handover and Codex Onboarding Reference

Source snapshot: version 1.5.0

Author: STYL15HH1

Target platform: Windows x64

Primary stack: Python 3.11+, Tkinter/ttk, Win32 APIs, PyInstaller, Inno Setup



STYL15HH1

Purpose

This document is a durable technical handover for a fresh Codex installation or another coding agent. It describes what the application is, how the current 1.5.0 source tree is organized, how data and UI flow through the program, what Windows-specific behavior must be preserved, how the application is built, and which known issue is currently pending. The source code remains the final authority if this document and the repository ever diverge.

1. Executive summary

IP Info Widget is a compact Windows desktop widget intended primarily for users who want immediate visibility of the public IP address currently exposed to the internet. It is particularly useful when switching VPN endpoints because the widget refreshes automatically and displays the current country, country flag, public IP, city, optional ISP, and optional ping result without opening a browser.

The application retrieves public IP and approximate geolocation data from <https://ipwho.is/> over HTTPS. Country flag images are bundled locally. It detects changes of the public IP during the current run, stores up to 50 changes locally, and can display a Windows toast notification when the IP changes.

Version 1.5.0 is modular. The former monolithic design has been split into core, ui, and windows layers. The Tkinter main thread owns all UI changes. Network refreshes execute in a daemon worker thread and return results through a queue.

The application supports installed mode and explicit portable mode. It also provides Compact, Normal, and Monitoring layouts, dynamic country icons in the taskbar and notification area, monitor pinning, themes, opacity, autostart, always-on-top, history, settings, and an About dialog.

Current important development issue

The widget can become invisible on the physical Windows desktop while the process remains active and visible on the taskbar. On the same computer it can become visible through an RDP session. Current source inspection shows that persisted x/y coordinates are restored without validating whether the widget rectangle still intersects an active monitor work area. In addition, `rebuild_layout()` treats negative x/y as invalid even though negative coordinates are legitimate in Windows multi-monitor topologies. The planned fix is automatic position validation plus a tray action named Restore widget to primary monitor. DPI should not be changed blindly; first use the existing Win32 monitor work-area coordinate system.

2. Project identity

Property	Current value
Application	IP Info Widget
Version	1.5.0
Author	STYL15HH1
Repository identity	github.com/STYL15HH1/ip-info-widget
License	MIT
Platform	Windows, installer targets x64-compatible
Language/UI	Python 3.11+ / Tkinter + ttk
Public IP service	https://ipwho.is/
Packaging	PyInstaller one-file, windowed
Installer	Inno Setup
Canonical version source	<code>core/version.py -> APP_VERSION</code>

3. Functional scope

- Public IP lookup: current public IP, country, ISO country code, city and ISP.
- Automatic refresh: default 5 seconds, validated to a minimum of 5 seconds.
- IP change detection: compares successful refreshes during the current process lifetime.

- History: stores the newest 50 change records locally in JSON.
- Notifications: optional Windows toast when the public IP changes.
- Ping: optional Windows ping to a configured host, default 1.1.1.1.
- Display modes: Compact, Normal and Monitoring.
- Visual settings: Dark/Light theme, opacity 55%-100%, optional city and ISP.
- Window behavior: draggable, borderless, always-on-top configurable, monitor pinning.
- Windows integration: notification-area tray, dynamic taskbar icon, autostart, monitor enumeration.
- Portable mode: explicit --portable switch with local config/history folders.
- About: current version, STYL15HH1 attribution, GitHub link, MIT information and bundled author avatar.

4. Source tree and responsibilities

```
ip-info-widget-1.5.0/
|-- app.py
|-- core/
|   |-- version.py
|   |-- models.py
|   |-- storage.py
|   |-- settings.py
|   |-- history.py
|   |-- ip_service.py
|   `-- ip_monitor.py
|-- ui/
|   |-- widget.py
|   |-- layouts.py
|   |-- settings_window.py
|   |-- history_window.py
|   |-- about_window.py
|   `-- menu.py
|-- windows/
|   |-- monitors.py
|   |-- tray.py
|   |-- taskbar.py
|   |-- notifications.py
|   `-- autostart.py
|-- assets/
|   |-- icons/
|   |-- author/
|   `-- images/flags/    (252 bundled PNG flags)
|-- docs/
|   |-- ARCHITECTURE.md
|   `-- screenshots/
|-- README.md
|-- CHANGELOG.md
|-- LICENSE
|-- requirements.txt
|-- build_exe.bat
`-- installer.iss
```

File/module	Responsibility
app.py	Bootstrap, PyInstaller Tcl/Tk environment, object composition, portable mode startup.
core/models.py	Immutable dataclasses: IPInfo, PingResult, IPChange, RefreshResult.
core/storage.py	Single path policy for installed/portable data and bundled resources.
core/settings.py	Versioned defaults, validation and JSON persistence. CONFIG_VERSION = 2.
core/history.py	Bounded local JSON history, max 50 entries.
core/ip_service.py	ipwho.is HTTPS lookup and Windows ping subprocess.
core/ip_monitor.py	Current-IP state, change detection and history insertion.
ui/widget.py	Main Tk controller, refresh queue, rendering, drag/position, tray coordination.
ui/layouts.py	Compact, Normal and Monitoring presentation only.

File/module	Responsibility
ui/settings_window.py	Settings dialog and user-editable preferences.
ui/history_window.py	History display and clearing.
ui/about_window.py	Themed single-instance About dialog.
ui/menu.py	Right-click widget context menu.
windows/monitors.py	EnumDisplayMonitors/GetMonitorInfoW work-area discovery.
windows/tray.py	pystray icon, show/hide/refresh/quit callbacks, country flag icon.
windows/taskbar.py	Native HICON creation/lifecycle and WM_SETICON/AppUserModelID handling.
windows/notifications.py	winotify toast for IP changes.
windows/autostart.py	HKCU Run registry integration.

5. Runtime architecture and data flow

```

IPService
|
v
IPMonitor
|
v
RefreshResult
|
+--> HistoryStore (only when public IP changes)
|
v
background worker thread
|
v
queue.Queue
|
v
IPInfoWidget on Tk main thread
|--> WidgetLayout
|--> TrayController
|--> TaskbarIcon
`--> Windows notification on IP change

```

The separation is intentional. IPService performs no UI work. IPMonitor owns only the previous public IP needed for reliable change detection. IPInfoWidget owns rendered UI state, timers, settings, drag state and queue processing. Layout classes receive DisplayData and decide only how to display it.

Threading invariant

Tkinter operations must stay on the Tk main thread. Background network work and pystray callbacks must communicate with the UI through the existing queue/event mechanism. A future agent must not call Tkinter widgets directly from a worker or tray thread.

6. Data model and network behavior

IPInfo: ip, country, country_code, city, isp. PingResult: reachable, latency_ms, host. IPChange: changed_at, old_ip, new_ip, country, country_code. RefreshResult: info, optional ping, optional change.

The public-IP endpoint is <https://ipwho.is/>. Each request adds a time-based query value to discourage caching and sends User-Agent IPInfoWidget/1.5.0 plus Cache-Control: no-cache. Timeout is 8 seconds. Invalid responses and transport/JSON errors are normalized to IPServiceError.

Ping uses the Windows command `ping -n 1 -w 1500`, with a subprocess timeout of 3 seconds and `CREATE_NO_WINDOW` when available. The latency parser recognizes English 'time' and Polish 'czas'.

The first successful refresh establishes `IPMonitor.current_ip` and does not create a history entry. A later successful refresh with a different IP creates an `IPChange` and writes it to `HistoryStore`.

7. Settings and persistence

Settings are JSON, configuration schema version 2. Current defaults from core/settings.py are:

```
{
  "config_version": 2,
  "theme": "dark",
  "opacity": 0.92,
  "refresh_seconds": 5,
  "show_location": true,
  "show_isp": false,
  "monitor_device": null,
  "ping_host": "1.1.1.1",
  "ping_enabled": false,
  "autostart": false,
  "always_on_top": true,
  "display_mode": "normal",
  "x": null,
  "y": null
}
```

Settings loading merges stored values over defaults, forces config_version to 2, validates display_mode, clamps refresh_seconds to at least 5 and opacity to 0.55-1.0. Malformed or missing settings fall back to defaults.

Normal mode stores settings at %LOCALAPPDATA%\MyIPWidget\settings.json and history at %LOCALAPPDATA%\MyIPWidget\ip-history.json. Portable mode stores config/settings.json and history/ip-history.json beside the executable/application folder. Portable startup forces autostart false.

8. UI modes and behavior

Compact: local country flag plus public IP only. Normal: country, ISO code, flag, public IP and optional details line. Monitoring: country, ISO code, flag, IP, city, ISP and actual ping status. Monitoring deliberately does not claim VPN, IPv6 or DNS state because those capabilities are not implemented.

The widget is borderless, draggable and can be pinned to a selected monitor. Clicking the visible IP copies it to the clipboard. Right-click opens the context menu with refresh, connection test, hide, history, settings, About and quit.

9. Windows integration

Monitor discovery

windows/monitors.py uses EnumDisplayMonitors and GetMonitorInfoW. It stores rcWork coordinates rather than the full rcMonitor rectangle, so placement can respect taskbar/reserved work areas. It also records the Windows display device name and primary-monitor flag.

Icons

The EXE, installer and shortcuts use assets/icons/ip-info-widget.ico. The notification-area tray starts with the application icon and switches to the current country flag after a successful lookup. The taskbar icon is handled separately by windows/taskbar.py using a process AppUserModelID, native HICON creation, SetClassLongPtrW and WM_SETICON. Previous native icon handles are released to avoid leaks.

Autostart and notifications

Autostart uses HKCU\Software\Microsoft\Windows\CurrentVersion\Run with value name 'IP Info Widget'. Portable mode disables it. IP-change notifications use winotify when available and fail silently if the notification subsystem is unavailable.

10. Build and release

requirements.txt contains Pillow>=10.0, pystray>=0.19,<0.20 and winotify>=1.1,<2.0. Tkinter comes from Python.

build_exe.bat reads APP_VERSION from core/version.py, installs build dependencies, then invokes PyInstaller in one-file/windowed mode. It explicitly bundles _tkinter.pyd, tcl86t.dll, tk86t.dll, Tkinter library data, Tcl/Tk runtime data, flags, author image and application icon. This explicit Tcl/Tk packaging is a preserved fix and should not be removed casually.

The build produces dist/IPInfoWidget-1.5.0.exe. If Inno Setup is available, installer.iss produces installer/IPInfoWidget-Setup-1.5.0.exe. The installer targets x64-compatible Windows, installs under Program Files, can create a desktop shortcut and uses the bundled application icon.

Release invariant

core/version.py is the canonical version source. When preparing a release, build output, installer metadata, README examples and CHANGELOG should remain consistent with that version.

11. Known issue pending fix: invisible widget

Observed behavior: the process runs and appears on the Windows taskbar, but the main widget can be invisible in the physical/local session. Connecting to the same machine with RDP can make the widget visible.

Source-level finding

ui/widget.py currently restores persisted x/y coordinates without validating that the resulting widget rectangle intersects any currently active monitor work area. This can leave the window off-screen after display topology, resolution, scaling or RDP changes. rebuild_layout() also contains a non-negative coordinate assumption, which is incorrect because Windows permits monitors to the left or above the primary display, producing valid negative x or y coordinates.

Planned focused remediation

- Extend windows/monitors.py with primary-monitor, rectangle visibility and safe-position helpers.
- Validate the actual widget rectangle against current rcWork monitor rectangles after layout sizing.
- Accept valid negative coordinates.
- If the saved position is invalid, move only the widget geometry to a safe position on the primary monitor and persist the repaired x/y.
- Add a tray command: Restore widget to primary monitor.
- Route tray recovery through the Tk main-thread queue.
- Do not change DPI-awareness mode or manually scale coordinates unless testing proves that DPI is the actual cause.

12. Engineering constraints for future work

- Source code is authoritative. Documentation must be updated when behavior changes.
- Prefer small, focused changes. Do not perform unrelated refactors during bug fixes.
- Keep Windows-specific APIs under windows/ and presentation-specific code under ui/.
- Do not grow app.py back into a monolith.
- Keep Tkinter mutations on the main thread.
- Preserve the working Tcl/Tk PyInstaller bundling unless a tested replacement exists.
- Do not fake unsupported status. VPN detection, IPv6 status, DNS leak checks, ASN/reputation and threat intelligence are not implemented in 1.5.0.
- New bundled assets must resolve correctly in source mode and PyInstaller one-file mode.

- Portable data paths and bundled resource paths are separate concepts and must remain separate.
- Do not claim tests were executed when they were not.

13. Fast onboarding procedure for Codex

For a fresh Codex project, place this PDF and the source tree in the project folder. Also place the companion CODEX_ONBOARDING_IP_INFO_WIDGET.md file at the repository root. Then start with the onboarding instruction below.

Read the project before modifying anything.

Priority order:

1. Read CODEX_ONBOARDING_IP_INFO_WIDGET.md.
2. Read this handover PDF if PDF reading is available.
3. Inspect README.md and CHANGELOG.md.
4. Inspect docs/ARCHITECTURE.md.
5. Inspect app.py.
6. Inspect all Python modules under core/, ui/ and windows/.
7. Inspect requirements.txt, build_exe.bat and installer.iss.
8. Compare documentation with source code.

Do not modify code yet.

Return:

- current version,
- architecture map,
- runtime/data flow,
- settings/storage model,
- Windows integrations,
- packaging model,
- known issue(s),
- any documentation/source inconsistencies.

If documentation conflicts with source, source code wins.

After Codex produces that report, give it one development task at a time. Require a short implementation plan before edits for non-trivial tasks, then require tests and a final changed-files report.

14. Regression checklist

- Application starts from source and from built EXE.
- Public IP lookup returns IP/country/code and optional city/ISP.
- Refresh timer and manual refresh work.
- IP changes create history entries, max 50.
- Notifications do not crash the app when unavailable.
- Ping works and does not block Tkinter UI.
- Compact, Normal and Monitoring layouts render correctly.
- Theme, opacity, city/ISP visibility and always-on-top persist.
- Dragging and monitor pinning work with multi-monitor layouts, including negative coordinates.
- Tray show/hide/refresh/quit and dynamic country flag work.
- Taskbar icon fallback and country-flag update work without leaking native handles.
- Normal and --portable storage remain isolated.
- Portable mode does not enable autostart.
- About window shows current APP_VERSION and STYL15HH1 identity.
- PyInstaller build still bundles Tcl/Tk, flags, icons and author asset.

- Installer version and EXE filename match core/version.py.

15. Source snapshot used for this handover

This handover was generated from the supplied archive ip-info-widget-1.5.0.zip, whose contained files are dated 2026-08-26. The analysis used the actual Python modules, README, CHANGELOG, architecture document, build script, installer script and bundled asset structure from that archive. No assumption was made that a public GitHub branch is newer than this snapshot.

The source archive itself should be preserved separately. This PDF intentionally documents structure and critical behavior rather than duplicating every source line. Codex should always inspect the real source tree before editing.